all the functions to be used are already written. We will be using the module named *sklearn* for the task.

```
from sklearn.linear_model import
LinearRegression
lm = LinearRegression()
lm.fit(X_train, y_train)
y_pred = lm.predict(X_test)
```

Here, *X_train* is the training input; *y_train* is training expected output; *X_test* is testing input; *y_test* is testing actual output; and *y_pred* is testing predicted output.

The methods used are:

- *LinearRegression():* This fits a linear model with coefficients B = (B1, …, BN) to minimise the residual sum of squares between the observed targets in the data set and the targets predicted by the linear approximation.
- *fit():* This method fits the model to the input training instances.
- *predict():* This performs predictions on the testing instances, based on the parameters learned during `fit`. The output source code is:

```
•   y_pred
    •   array([ 30.11355599,
        139.04715128, 220.74734774,
        338.75874263,402.30333988,
        538.47033399, 665.55952849])

•   print("Coeffient(B1) {} and
    Intercept(B0) {}".format(lm.coef_,
    lm.intercept_))
    •  Coeffient(B1) [90.77799607]
       and Intercept(B0)
       30.11355599214147
```

Predictions

Now that we have trained our model, let's see how close to the truth the predictions are.

```
plt.title('Prediction of Price of Wine
based on the given Age')
plt.xlabel("Age of wine") #x label
plt.ylabel("Price of wine") #y label
```

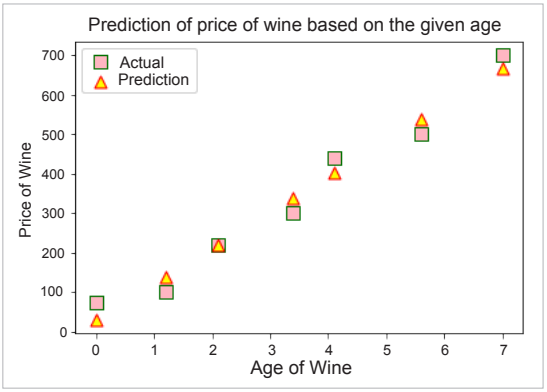

Figure 8: Prediction vs actual relationship

```
plt.scatter(X_test, y_test, c ="pink",
            marker ="s",
            edgecolor ="green",
            s = 100,
            label = "Actual")
plt.scatter(X_test, y_pred, c
="yellow",
            marker ="^",
            edgecolor ="red",
            s = 80,
            label = "Prediction")
plt.legend()
```

Though the model performed really well, the predictions were not completely accurate. So how confident are we about our model's prediction? To answer this question, we need to know the 'confidence interval'.

Generally, we don't want our model to be 100 per cent accurate, because it will be considered as an overfit and will not generalise well.

In simple terms, assume the model is a student, who is going for an English language exam. We have given this student a few questions to learn which, coincidently, are asked in the exam. The model does well, scoring full marks.

However, this will not always be the case. If the questions are different, the model will not understand them and fail, as it has memorised only these answers verbatim.

The takeaway from here is that instead of giving a few questions, giving the whole book (i.e., more data) to the model will make memorising hard, and it will start learning and finding patterns.

The assumptions we make here are:

- Input and output variables should be numeric.
- Outliers should be removed or replaced.
- Remove collinearity between input variables (x), as linear regression will over-fit your data when you have highly correlated input variables. There should be collinearity with input and output but not within input variables.

Rescale inputs, as linear regression will often make more reliable predictions if you rescale input variables using standardisation or normalisation. END

References

[1] *https://towardsdatascience.com/introduction-to-machine-learning-algorithms-linear-regression-14c4e325882a*
[2] *https://machinelearningmastery.com/linear-regression-for-machine-learning/*
[3] *https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols.html#sphx-glr-auto-examples-linear-model-plot-ols-py*
[4] *https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html*
[5] The Manga Guide to Regression Analysis

By: Thangaselvi Arichandrapandian

The author works in a leading bank as AVP. She is a perennial learner and loves the quote: "The more I learn, the more I realise how much I don't know."
- *Albert Einstein.*